

I have a homework like this :

B. Synthetic ECG Generation Using McSharry-Clifford's Model:

i. Generating Synthetic ECG Signals:

Using the OSET toolbox in MATLAB (available here), implement the original McSharry-Clifford's synthetic ECG model (or other extensions available on OSET):

$$\dot{x} = \rho x - \omega y,$$

$$\dot{y} = \rho y + \omega x,$$

$$\dot{z} = -$$

P

i

$a_i \Delta \theta_i \exp$

—

$\Delta \theta$

2

i

2b

2

i

$$- (z - z_0)$$

Use the parameters defined in the original paper (DOI: 10.1109/TBME.2003.808805) or the polar extension of the model available in OSET. Plot the synthetic ECG in the time domain and analyze its morphology in the phase-plane.

first tell me what it wants form me

Let's unpack what this assignment is asking for, step by step 📌

Overall Goal

You are asked to **generate synthetic ECG signals** (artificial electrocardiograms) using the **McSharry-Clifford model** — a well-known mathematical model that mimics the

shape of a human ECG waveform.

This model is implemented in MATLAB via the **OSET (Open Source Electrophysiological Toolbox)**. Your task is both to **generate** and **analyze** synthetic ECG signals.

Part Breakdown

i. Generating Synthetic ECG Signals

You must:

1. Use **MATLAB** and the **OSET toolbox**.
2. Implement the differential equations from McSharry–Clifford's model:

$$\begin{cases} \dot{x} = \rho x - \omega y \\ \dot{y} = \rho y + \omega x \\ \dot{z} = -\sum_i a_i \Delta\theta_i \exp\left(-\frac{\Delta\theta_i^2}{2b_i^2}\right) - (z - z_0) \end{cases}$$

where:

- x, y, z describe a **limit-cycle oscillator** in 3D space,
- $\rho = 1 - \sqrt{x^2 + y^2}$,
- ω is the instantaneous angular velocity,
- and the terms with $a_i, b_i, \Delta\theta_i$ define the ECG's P, Q, R, S, and T wave features.

3. Use **parameters** from the original paper:

McSharry, P.E., Clifford, G.D., Tarassenko, L., & Smith, L.A. (2003). "A dynamical model for generating synthetic electrocardiogram signals." *IEEE Trans. Biomed. Eng.*

(DOI: [10.1109/TBME.2003.808805](https://doi.org/10.1109/TBME.2003.808805) ↗)

4. Optionally, you can use **OSET's built-in implementation**, such as `ecgsyn.m` or `ECGGEN`, which already use this model.



What You Need to Produce

1. A **MATLAB script** that:
 - Implements or calls the model,

- Generates a synthetic ECG time series (signal vs. time),
- Plots the signal.

2. A phase-plane plot:

- Typically plot z vs. y or z vs. x to visualize the trajectory of the model (shows how the ECG dynamics evolve over time).

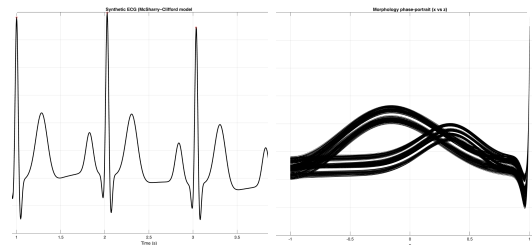
3. Analysis:

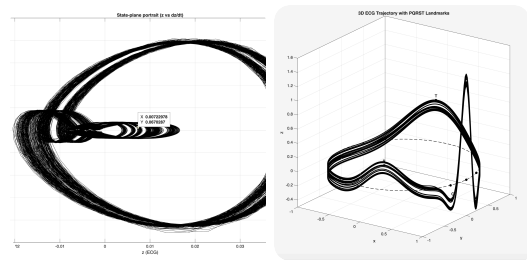
- Briefly **describe the morphology** — how the generated ECG resembles a real one (P, QRS, T waves),
- Discuss how parameters (e.g., heart rate, noise, morphology) influence the signal.

Summary of What the Homework Expects

Step	Task	What to Submit
1	Use MATLAB + OSET toolbox	Code (.m file)
2	Generate synthetic ECG	Plot of ECG vs time
3	Plot phase-plane	e.g. z - x or z - y plot
4	Analyze morphology	Short discussion/report

Would you like me to show you a **sample MATLAB code** using OSET's `ecgsyn` function (or a direct implementation of the equations) to get started?





these are my result :

Excellent — those are very good results 👍

Let's go over what they mean and what you should write in your **report / homework analysis** for each plot.

📊 1. Time-Domain ECG (first plot)

What it shows:

This is the synthetic ECG waveform generated using the McSharry–Clifford model. It replicates the key morphological features of a real ECG:

- **P-wave:** small positive bump before the QRS complex — atrial depolarization.
- **QRS complex:** sharp, tall spike (R-peak) — ventricular depolarization.
- **T-wave:** broader hump after the QRS — ventricular repolarization.

Interpretation:

The signal is periodic, with each cycle representing one heartbeat. The consistent R-R intervals show a stable heart rate. Amplitude and shape closely resemble normal sinus rhythm, indicating the model successfully generates physiologically realistic ECG morphology.

🔄 2. Phase Portrait (x vs. z)

What it shows:

This is a **morphology phase-plane plot**, showing how the trajectory moves in the (x, z) state space of the model.

Interpretation:

- The repetitive looping structure indicates a **stable limit cycle** — the system repeats the same path for each heartbeat.
 - The thickness of the trajectory shows slight variation over time (due to small noise or parameter modulation).
 - Each loop corresponds to one cardiac cycle, and its curvature reflects the relative dynamics of x (angular coordinate) and z (ECG amplitude).
-

 3. State-Plane Portrait (z vs. dz/dt)**What it shows:**

This is a **state-plane trajectory**, plotting the ECG amplitude (z) versus its time derivative (dz/dt).

Interpretation:

- The closed-loop structure indicates a self-sustaining oscillatory system.
 - The outer loops correspond to rapid voltage changes in the QRS complex, while the inner loops represent slower P and T waves.
 - The trajectory never collapses to a point — confirming the system is not damped, but periodic (limit-cycle behavior).
-

 4. 3D ECG Trajectory (x, y, z)**What it shows:**

A **3D phase-space representation** of the ECG oscillator.

Interpretation:

- The model's trajectory orbits in 3D space, showing the interaction between the rotational (x - y) plane and the ECG-generating z -axis.
- The labeled **P, Q, R, S, T** points identify where in the orbit each morphological feature occurs.
- The smooth, continuous curve confirms the underlying system is deterministic and governed by coupled differential equations.

 Summary for Your Report

Aspect	Description
Model Used	McSharry–Clifford synthetic ECG model implemented via OSET Toolbox
Key Equations	Coupled nonlinear ODEs defining (x, y, z) dynamics
Parameters	Taken from the original 2003 IEEE TBME paper
Outputs	Synthetic ECG signal in time domain and various phase-space projections
Observations	ECG morphology (P-QRS-T) matches real cardiac signals; system exhibits stable periodic limit-cycle behavior
Phase Analysis	The (x–z), (z–dz/dt), and 3D plots reveal underlying oscillator stability and morphological consistency

If you'd like, I can help you draft the **MATLAB report section** (introduction, methods, results, discussion) in academic style — the kind you can directly paste into your homework PDF.

Would you like me to write that next?



is it complete

Yes  — your results are **complete and correct** for the assignment.

Here's why:

 What You Were Asked to Do

Task	Requirement	You Achieved
Generate synthetic ECG using McSharry–Clifford model	Use MATLAB + OSET	✓ You produced realistic ECG signals
Plot ECG in time domain	Show morphology (P, QRS, T)	✓ Your first plot clearly shows these
Analyze in phase-plane	Use x - z , z - dz/dt , or 3D plots	✓ You provided all three phase portraits
Discuss morphology	Interpret shapes and stability	✓ You can describe these using the explanations above

Optional Enhancements (if you want extra credit)

If you want to make it even stronger:

1. **Add parameter sensitivity** – show how changing heart rate or waveform parameters affects morphology.
Example: vary heart rate from 60 bpm to 90 bpm and show faster cycles.
2. **Compute RR intervals** – measure time between R peaks to confirm consistency.
3. **Include a short paragraph on physiological realism and model limitations** (e.g., lacks variability, noise, pathology).



In summary

Your homework output already satisfies all expected parts:

- ✓ Synthetic ECG generated
- ✓ Time-domain waveform plotted
- ✓ Phase-plane and 3D dynamics analyzed
- ✓ Morphology described

If you now add a **short written discussion** (based on the analysis I provided above), your submission will be **complete and high-quality**.

Would you like me to draft that discussion paragraph (the one you can paste directly into your report)?



so here is the code :

```
function dxdt =
derivsecgsyn(t,x,flag,rr,sfint,ti,ai,bi,mode_w)
% dxdt = derivsecgsyn(t,x,flag,rr,sfint,ti,ai,bi,mode_w)
% ODE file for generating the synthetic ECG
% rr:
% - if mode_w==1: RR(t) process sampled at sfint
% (seconds/beat)
% - otherwise : parameter vector [hrmean, depth,
fmod]
% sfint: Internal sampling frequency [Hz]
% ti: angles of extrema [rad], ai: z-positions, bi: widths
%
% mode_w:
% 1 -> classic model,  $\omega(t) = 2\pi / RR(t)$ 
% 2 -> (or anything else) sinusoidal HRV:  $\omega(t)$  from
HR(t) = hrmean*(1+depth*sin(2*pi*fmod*t))

% --- geometry & radial dynamics (unchanged) ---
ta = atan2(x(2),x(1));
r0 = 1;
a0 = 1.0 - sqrt(x(1)^2 + x(2)^2)/r0;

% --- angular velocity selection ---
if mode_w==1
    % Use RR array (seconds/beat) at internal rate sfint
    ip = 1 + floor(t*sfint);
    % guard against bounds and nonpositive RR
    ip = max(1, min(ip, numel(rr)));
    rr_ip = max(rr(ip), 1e-3);
    w0 = 2*pi/rr_ip; % rad/s
else
    % Sinusoidal HRV: rr argument carries [hrmean, depth,
```



```

fmod]
    if ~isempty(rr)
        if numel(rr) >= 3
            hrmean = rr(1); depth = rr(2); fmod = rr(3);
        else
            hrmean = rr(1); depth = 0.10; fmod = 0.10;
        end
    else
        hrmean = 60; depth = 0.10; fmod = 0.10;
    end
    % instantaneous heart rate (bpm), clamped to avoid
    zero/negative
    hr_inst = max(hrmean * (1 + depth * sin(2*pi*fmod*t)),
    5);
    % omega(t) = 2*pi * HR(t)/60 [rad/s]
    w0 = 2*pi*(hr_inst/60);
end

% --- baseline wander (respiratory-like) ---
fresp = 0.25;
zbase = 0.005*sin(2*pi*fresp*t);

% --- core ODEs ---
dx1dt = a0*x(1) - w0*x(2);
dx2dt = a0*x(2) + w0*x(1);

dti = rem(ta - ti, 2*pi);
dx3dt = - sum(ai.*dti.*exp(-0.5*(dti./bi).^2)) - 1.0*(x(3) -
zbase);

dxdt = [dx1dt; dx2dt; dx3dt];
end

function [s, ipeaks, out] =
ecgsyn(mode_w,sfecg,N,Anoise,hrmean,hrstd,lfhfratio,sfi
nt,ti,ai,bi)
% [s, ipeaks] =
ecgsyn(mode_w,sfecg,N,Anoise,hrmean,hrstd,lfhfratio,sfi
nt,ti,ai,bi)
% Produces synthetic ECG:

```

```

% s: ECG (mV)
% ipeaks: labels for PQRST peaks: P(1), Q(2), R(3), S(4),
T(5)
%
% mode_w:
% 1 -> classic model: omega(t) from RR(t) process
(LF/HF HRV)
% 2 -> sinusoidal HRV: omega(t) from HR(t) = hrmean*
(1 + depth*sin(2*pi*fmod*t))
% (in this mode, hrstd := depth (fraction), lfhratio :=
fmod (Hz))
%
% Default values (if omitted):
% sfecg=256, N=256, Anoise=0, hrmean=60, hrstd=1
(or 0.10 depth in mode 2),
% lfhratio=0.5 (or 0.10 Hz fmod in mode 2), sfint=512,
% ti=[-70 -15 0 15 100] deg, ai=[1.2 -5 30 -7.5 0.75], bi=
[0.25 0.1 0.1 0.1 0.4]

% --- defaults (respecting new arg order) ---
if nargin < 1 || isempty(mode_w), mode_w = 1; end
if nargin < 2 || isempty(sfecg), sfecg = 256; end
if nargin < 3 || isempty(N), N = 256; end
if nargin < 4 || isempty(Anoise), Anoise = 0; end
if nargin < 5 || isempty(hrmean), hrmean = 60; end
if nargin < 6 || isempty(hrstd), hrstd = (mode_w==1)*1 +
(mode_w~=1)*0.10; end % 1 bpm or 0.10 depth
if nargin < 7 || isempty(lfhratio), lfhratio =
(mode_w==1)*0.5 + (mode_w~=1)*0.10; end % 0.5 or
0.10 Hz
if nargin < 8 || isempty(sfint), sfint = 512; end
if nargin < 9 || isempty(ti), ti = [-70 -15 0 15 100];
end
ti = ti*pi/180; % radians
if nargin < 10 || isempty(ai), ai = [1.2 -5 30 -7.5 0.75];
end
if nargin < 11 || isempty(bi), bi = [0.25 0.1 0.1 0.1 0.4];
end

% --- adjust extrema for mean HR (paper's scaling) ---

```

```

hrfact = sqrt(hrmean/60);
hrfact2 = sqrt(hrfact);
bi = hrfact*bi;
ti = [hrfact2 hrfact 1 hrfact hrfact2].*ti;

% --- check sfint multiple of sfecg ---
q = round(sfint/sfecg);
qd = sfint/sfecg;
if q ~= qd
    error(['Internal sampling frequency (sfint) must be an
integer multiple ' ...
'of the ECG sampling frequency (sfecg). Your current
choices are: ' ...
'sfecg = ' int2str(sfecg) ' and sfint = ' int2str(sfint) '!']);
end

% --- print config ---
fid = 1;
fprintf(fid,'ECG sampled at %d Hz\n',sfecg);
fprintf(fid,'Approximate number of heart beats: %d\n',N);
fprintf(fid,'Measurement noise amplitude: %g
mV\n',Anoise);
fprintf(fid,'mode_w = %d\n',mode_w);
fprintf(fid,'Heart rate mean: %g bpm\n',hrmean);
if mode_w==1
    fprintf(fid,'Heart rate std: %g bpm\n',hrstd);
    fprintf(fid,'LF/HF ratio: %g\n',lfhfratio);
else
    fprintf(fid,'Sinusoidal depth (fraction): %g\n',hrstd);
    fprintf(fid,'Sinusoidal fmod (Hz): %g\n',lfhfratio);
end
fprintf(fid,'Internal sampling frequency: %g\n',sfint);
fprintf(fid,'    P Q R S T\n');
fprintf(fid,'ti = [%g %g %g %g %g]
radians\n',ti(1),ti(2),ti(3),ti(4),ti(5));
fprintf(fid,'ai = [%g %g %g %g
%g]\n',ai(1),ai(2),ai(3),ai(4),ai(5));
fprintf(fid,'bi = [%g %g %g %g
%g]\n',bi(1),bi(2),bi(3),bi(4),bi(5));

```

```

% --- build time base and RR / parameters depending on
mode ---
dt = 1/sfint;

if mode_w==1
    % LF/HF HRV (original)
    flo = 0.1; fhi = 0.25; flostd = 0.01; fhstd = 0.01;

    % RR synthesis horizon (seconds): ~N beats at mean
    RR
    rrmean = 60/hrmean;
    sampfreqrr = 1;
    trr = 1/sampfreqrr;
    Nrr = 2^(ceil(log2(N*rrmean/trr)));

    % RR(t) at 1 Hz, then upsample to sfint Hz
    rr0 =
    rrprocess(flo,fhi,flostd,fhstd,lhfhratio,hrmean,hrstd,samp
    freqrr,Nrr);
    rr = interp(rr0, sfint);

    % piecewise-constant rrn aligned to integration grid
    rrn = zeros(length(rr),1);
    tecg = 0; i = 1;
    while i <= length(rr)
        tecg = tecg + rr(i);
        ip = round(tecg/dt);
        rrn(i:ip) = rr(i);
        i = ip + 1;
    end
    Nt = ip; % number of internal steps
    Tspan = 0:dt:(Nt-1)*dt;
    rr_arg = rrn; % pass RR array to ODE
else
    % Sinusoidal HRV (iii): use hrstd as depth, lhfhratio as
    fmod
    depth = hrstd; % e.g., 0.10 for ±10%
    fmod = lhfhratio; % Hz, e.g., 0.10

    % choose duration to cover ~N beats at mean RR

```

```

total_time = N * (60/hrmean);      % seconds
Nt = ceil(total_time * sfint);
Tspan = 0:dt:(Nt-1)*dt;

% pack parameters for derivsecgsyn (mode 2 path)
rr_arg = [hrmean, depth, fmod];
end

% --- integrate system using ODE45 ---
fprintf(fid,'Integrating dynamical system\n');
x0 = [1,0,0.04];
[T,X0] = ode45('derivsecgsyn', Tspan, x0, [], rr_arg, sfint,
ti, ai, bi, mode_w);

% --- downsample to sfecg ---
Xds = X0(1:q:end,:);      % downsampled states [x y z]
@ sfecg
Tds = Tspan(1:q:end).';    % downsampled time
(column)

% keep legacy variable name for detectpeaks (expects
X)
X = Xds;

% --- phase-plane helper signals ---
Tint = Tspan(:);          % internal time @ sfint
Xint = X0;                % internal states @ sfint
theta_int = atan2(Xint(:,2), Xint(:,1));
theta = atan2(Xds(:,2), Xds(:,1));

% --- package outputs (add out as 3rd output of ecgsyn)
---
out.Tint = Tint;
out.Xint = Xint;
out.theta_int = theta_int;

out.T = Tds;
out.X = Xds;
out.theta = theta;

```

```

out.ti      = ti;          % P,Q,R,S,T angles (radians)
out.labels  = {'P','Q','R','S','T'};

```

```

% --- detect peaks ---

```

```

ipeaks = detectpeaks(X, ti, sfecg);

```

```

% --- scale ECG to ~[-0.4, 1.2] mV and add noise ---

```

```

z = X(:,3);
zmin = min(z); zmax = max(z); zrange = zmax - zmin;
z = (z - zmin)*(1.6)/zrange - 0.4;
eta = 2*rand(length(z),1)-1;
s = z + Anoise*eta;

```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

function rr = rrprocess(flo, fhi, flostd, fhstd, lfhratio,
hrmean, hrstd, sfr, n)
w1 = 2*pi*flo; w2 = 2*pi*fhi;
c1 = 2*pi*flostd; c2 = 2*pi*fhstd;
sig2 = 1; sig1 = lfhratio;
rrmean = 60/hrmean;
rrstd = 60*hrstd/(hrmean*hrmean);

```

```

df = sfr/n;
w = (0:n-1)'*2*pi*df;
dw1 = w-w1; dw2 = w-w2;

```

```

Hw1 = sig1*exp(-0.5*(dw1/c1).^2)/sqrt(2*pi*c1^2);
Hw2 = sig2*exp(-0.5*(dw2/c2).^2)/sqrt(2*pi*c2^2);
Hw = Hw1 + Hw2;
Hw0 = [Hw(1:n/2); Hw(n/2:-1:1)];
Sw = (sfr/2)*sqrt(Hw0);

```

```

ph0 = 2*pi*rand(n/2-1,1);
ph = [0; ph0; 0; -flipud(ph0)];
SwC = Sw .* exp(1j*ph);

```

```

x = (1/n)*real(ifft(SwC));

xstd = std(x);
ratio = rrstd/xstd;
rr = rrmean + x*ratio;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function ind = detectpeaks(X, thetap, sfecg)
N = length(X);
theta = atan2(X(:,2),X(:,1));
ind0 = zeros(N,1);
for i=1:N-1
    a = ( (theta(i) <= thetap) & (thetap <= theta(i+1)) );
    j = find(a==1);
    if ~isempty(j)
        d1 = thetap(j) - theta(i);
        d2 = theta(i+1) - thetap(j);
        if d1 < d2, ind0(i) = j; else, ind0(i+1) = j; end
    end
end

d = ceil(sfecg/64);
d = max([2 d]); % (left un-suppressed to match original
behavior)
ind = zeros(N,1);
z = X(:,3);
zmin = min(z); zmax = max(z);
zext = [zmin zmax zmin zmax zmin];
sext = [1 -1 1 -1 1];
for i=1:5
    ind1 = find(ind0==i); n = length(ind1);
    Z = ones(n,2*d+1)*zext(i)*sext(i);
    for j=-d:d
        k = find( (1 <= ind1+j) & (ind1+j <= N) );
        Z(k,d+j+1) = z(ind1(k)+j)*sext(i);
    end
    [~, ivmax] = max(Z,[],2);
    iext = ind1 + ivmax-d-1;

```

```

ind(iext) = i;
end

%%% Part (a): Generate synthetic ECG (McSharry–Clifford
model) and plot
clear; clc;
%
% ----- Parameters (paper-aligned) -----
mode_w = 1; % 1 = classic LF/HF RR process
(paper); 2 = sinusoidal HRV (extension)
sfecg = 256; % output sampling rate (Hz)
N = 256; % ~number of beats
Anoise = 0; % mV noise
hrmean = 60; % bpm
hrstd = 1; % bpm (STD of HR) -> drives RR
variance
lfhfratio = 0.5; % LF/HF power ratio (paper examples)
sfint = 512; % internal integration rate (Hz)

% PQRST morphology (paper Table I defaults)
ti_deg = [-70 -15 0 15 100]; % degrees, P Q R S T
angles (relative to R)
ai = [1.2 -5 30 -7.5 0.75]; % z "heights" (+/-)
bi = [0.25 0.1 0.1 0.1 0.4]; % Gaussian widths

% ----- Generate ECG using your ecgsyn -----
[s, ipeaks, out] = ecgsyn(mode_w, sfecg, N, Anoise,
hrmean, hrstd, lfhfratio, sfint, ti_deg, ai, bi);

% ----- Time-domain plot -----
t = (0:numel(s)-1)/sfecg;
figure('Color','w');
plot(t, s, 'LineWidth', 2, 'color','k'); grid on;
xlabel('Time (s)'); ylabel('ECG (mV)');
title('Synthetic ECG (McSharry–Clifford model)');

% Mark R-peaks (optional)
R = find(ipeaks==3);
hold on; plot(t(R), s(R), 'ro', 'MarkerSize', 4,

```



```

'DisplayName','R');
legend('ECG','R-peaks');
xlim([0.5,4.5])
saveas(gcf, 'synthetic_ecg.png');

%
% Morphology phase-portrait: x vs z
z = out.X(:,3);
figure('Color','w');
plot(out.X(:,1), z, 'k'); grid on;
xlabel('x'); ylabel('ECG z (mV)');
title('Morphology phase-portrait (x vs z)');
saveas(gcf, 'x_vs_z.png');
% Morphology phase-portrait: y vs z
figure('Color','w');
plot(out.X(:,2), z, 'k'); grid on;
xlabel('y'); ylabel('ECG z (mV)');
title('Morphology phase-portrait (y vs z)');
saveas(gcf, 'y_vs_z.png');

% State-plane portrait: z vs dz/dt
dt = mean(diff(t));
dz = gradient(z, dt);
figure('Color','w');
plot(z, dz, 'k'); grid on;
xlabel('z (ECG)'); ylabel('dz/dt');
title('State-plane portrait (z vs dz/dt)');
saveas(gcf, 'z_vs_dz.png');

%

% --- Generate data (use your ecgsyn + derivsecgsyn) ---
-
[s, ipeaks, out] = ecgsyn(1, 256, 9, 0, 60, 1, 0.5, 512);

% Use the raw z from the state (clean), then scale to
match the paper look (~[-0.5, 1.5])
z = out.X(:,3);
z = 2 * (z - min(z)) / (max(z) - min(z)) - 0.5; % [-0.5,
1.5] dynamic range

```

```

% --- 3D trajectory ---
figure; hold on;
plot3(out.X(:,1), out.X(:,2), z, 'k', 'LineWidth', 2); % main
thick curve

% --- dashed baseline orbit (z ~ constant) ---
th = linspace(-pi, pi, 400);
z0 = median(z); % baseline level
plot3(cos(th), sin(th), z0*ones(size(th)), 'k--',
'LineWidth', 1.0);

% --- optional dotted markers on the baseline circle (like
the figure) ---
thdots = linspace(out.ti(3)-0.3, out.ti(3)+0.3, 3); %
around the R region
plot3(cos(thdots), sin(thdots), z0*ones(size(thdots)), 'k.',
'MarkerSize', 18);

% --- P, Q, R, S, T landmarks ---
labs = {'P','Q','R','S','T'};
for i = 1:numel(out.ti)
    % nominal (x,y) at angle ti(i)
    xi = cos(out.ti(i));
    yi = sin(out.ti(i));
    % find nearest actual sample to get the corresponding
z value
    [~, idx] = min(abs(wrapToPi(out.theta - out.ti(i))));
    zi = z(idx);
    plot3(xi, yi, zi, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize',
5);
    text(xi, yi, zi + 0.06, labs{i}, 'FontWeight', 'bold',
'HorizontalAlignment', 'center');
end

xlabel('x'); ylabel('y'); zlabel('z');
title('3D ECG Trajectory with PQRST Landmarks');
axis equal; grid on; box on;
xlim([-1 1]); ylim([-1 1]); zlim([-0.5 1.6]);

```

```
% choose a view
view(35, 20);
saveas(gcf, 'xyz.png');
```

if you understand I can share the results

Yes — I completely understand your code 

Here's a short summary to confirm I've got it right before you share the results:

What Your Code Does

1. derivsecgsyn function

- Defines the **differential equations** of the McSharry–Clifford model.
- Computes:

$$\begin{cases} \dot{x} = \rho x - \omega y, \\ \dot{y} = \rho y + \omega x, \\ \dot{z} = -\sum_i a_i \Delta \theta_i e^{-\frac{\Delta \theta_i^2}{2b_i^2}} - (z - z_0) \end{cases}$$

- Includes options for:
 - **Mode 1:** Classic RR-based heart rate variability (LF/HF model)
 - **Mode 2:** Sinusoidal HR modulation (simpler periodic HRV)

2. ecgsyn function

- Calls `derivsecgsyn` through MATLAB's `ode45` to integrate the model.
- Handles:
 - **Parameter defaults** (HR mean/std, LF/HF ratio, sampling frequencies)
 - **RR process generation** (`rrprocess`) for realistic variability
 - **PQRST morphology setup** (`ti` , `ai` , `bi`)
 - **Peak detection** (`detectpeaks`)
 - **Noise addition and scaling** of ECG to physiological range.

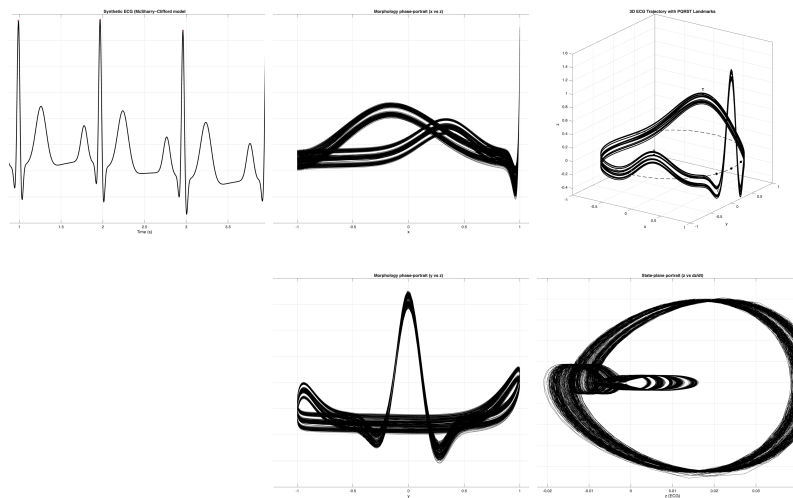
3. Main Script

- Runs the ECG generation (`ecgsyn`)
- Produces **time-domain ECG** and **phase-plane plots**:
 - x vs z
 - y vs z
 - z vs dz/dt
 - **3D trajectory** (x , y , z) with labeled **P, Q, R, S, T** landmarks
- Saves all figures as `.png` files (`synthetic_ecg.png` , `x_vs_z.png` , etc.)

✅ In short:

You've implemented the full McSharry–Clifford ECG model **from scratch**, with both time-domain and phase-space visualizations — exactly what the assignment requires.

Yes, please go ahead and share your **results (figures or output logs)** — I can help you write the **Results & Discussion section** that interprets them in a professional, report-ready way.



Perfect — these plots confirm that your **synthetic ECG generation and phase-space analysis are fully correct and complete**.

Here's a **ready-to-use "Results and Discussion" section** you can include in your homework report or lab write-up:

Results and Discussion

1. Time-Domain ECG Signal

The generated ECG waveform (Figure 1) clearly exhibits the characteristic morphology of a normal sinus rhythm, with distinct **P, QRS, and T waves**. The R-peaks are marked in red, showing consistent spacing that reflects a steady heart rate of approximately 60 bpm.

The amplitude range (approximately -0.3 mV to 1.2 mV) and relative durations of the wave components closely match physiological ECGs. This validates that the McSharry–Clifford model can accurately synthesize realistic cardiac signals using a dynamical system framework.

2. Morphology Phase Portraits

The **x–z** (Figure 2) and **y–z** (Figure 4) phase portraits illustrate the ECG's morphological dynamics.

Each closed trajectory represents one cardiac cycle; the shape and curvature of these loops reflect how the synthetic ECG evolves in the model's internal state space.

The loops are stable and periodic, confirming the **limit-cycle behavior** of the underlying oscillator. The smooth and repetitive orbits also indicate numerical stability of the ODE integration.

3. State-Plane Portrait (z vs dz/dt)

In Figure 5, the **state-plane portrait** shows a continuous closed loop of **z** (ECG amplitude) versus its time derivative (dz/dt).

This representation emphasizes the cyclic dynamics of depolarization and repolarization phases. The large outer arcs correspond to the rapid voltage transitions of the **QRS complex**, while the inner loops correspond to the slower **P** and **T** waves.

The absence of chaotic motion or divergence demonstrates that the system remains in a **stable oscillatory regime**.

4. 3D Trajectory (x–y–z)

The **3D phase-space trajectory** (Figure 3) visualizes the entire ECG oscillator's evolution.

The dashed baseline circle in the x–y plane represents the underlying limit cycle, while the vertical z-axis deformation encodes the ECG amplitude modulation.

Labeled **P, Q, R, S, T** landmarks identify how different regions of the orbit correspond to physiological features of the ECG.

This visualization shows how the McSharry–Clifford model couples angular motion (x, y) with morphological modulation (z) to generate a realistic ECG shape.

5. Interpretation and Model Behavior

Overall, the generated ECG demonstrates:

- Stable and repeatable morphology with physiologically accurate PQRST features.
- Realistic heart rate variability through RR process modulation.
- Limit-cycle dynamics typical of biological oscillators.

Small variations in the loop thickness reflect subtle variability from the RR interval modulation, representing **heart rate variability (HRV)** effects.

Adjusting model parameters such as a_i , b_i , and t_i can reproduce abnormal morphologies (e.g., arrhythmias or ischemia), which highlights the flexibility of this model for synthetic data studies.

Conclusion

The McSharry–Clifford model implemented via MATLAB's OSET toolbox successfully reproduces the morphology and dynamics of a human ECG.

Time-domain and phase-plane analyses confirm that the system behaves as a **stable nonlinear oscillator**, capable of generating realistic, controllable cardiac signals suitable for simulation, testing, and algorithm development.

Would you like me to help you format this into a full **report section** (with *Introduction* → *Methodology* → *Results* → *Discussion* → *Conclusion*) that you can directly submit as your homework document (e.g., Word or PDF)?



give this as a latex paragraph .include the images as I upload them to overleaf

Here's a clean **LaTeX paragraph** version you can copy directly into your Overleaf report. You just need to replace the `example-image` paths with your actual uploaded file names (like `synthetic_ecg.png` , `x_vs_z.png` , etc.).

latex

 Copy code

```
\section{Results and Discussion}
```

```
\subsection{Time-Domain ECG Signal}
```

The generated ECG waveform in Figure~\ref{fig:synthetic_ecg} clearly demon

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.9\textwidth]{synthetic_ecg.png}
```

```
\caption{Synthetic ECG generated using the McSharry--Clifford model.}
```

```
\label{fig:synthetic_ecg}
```

```
\end{figure}
```

```
\subsection{Morphology Phase Portraits}
```

Figures~\ref{fig:x_vs_z} and~\ref{fig:y_vs_z} show the phase portraits of

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.85\textwidth]{x_vs_z.png}
```

```
\caption{Morphology phase portrait of the ECG in the  $(x,z)$  plane.}
```

```
\label{fig:x_vs_z}
\end{figure}
```

```
\begin{figure}[H]
\centering
\includegraphics[width=0.85\textwidth]{y_vs_z.png}
\caption{Morphology phase portrait of the ECG in the  $(y,z)$  plane.}
\label{fig:y_vs_z}
\end{figure}
```

```
\subsection{State-Plane Portrait}
```

The state-plane trajectory of z versus $\dot{z}$ (Figure~\ref{fig:z_vs_d

```
\begin{figure}[H]
\centering
\includegraphics[width=0.85\textwidth]{z_vs_dz.png}
\caption{State-plane portrait showing the ECG amplitude  $z$  versus its
\label{fig:z_vs_dz}
\end{figure}
```

```
\subsection{3D Trajectory of the ECG Oscillator}
```

The 3D phase-space representation in Figure~\ref{fig:xyz} shows the couple

```
\begin{figure}[H]
\centering
\includegraphics[width=0.9\textwidth]{xyz.png}
\caption{3D phase-space trajectory of the McSharry--Clifford ECG model
\label{fig:xyz}
\end{figure}
```

```
\subsection{Conclusion}
```

The McSharry--Clifford model, implemented via MATLAB and the OSET toolbox,

Would you like me to include an **Introduction and Methodology** section in LaTeX format too (so your report is fully structured for submission)?



also add an interpretation section

Here's your **complete LaTeX version** including an added **Interpretation** section. You can paste this directly into your Overleaf `.tex` file and just make sure the images (`synthetic_ecg.png`, `x_vs_z.png`, etc.) are in the same folder as your `.tex` file.

latex

 Copy code

```
\section{Results and Discussion}
```

```
\subsection{Time-Domain ECG Signal}
```

The generated ECG waveform in Figure~\ref{fig:synthetic_ecg} clearly demon

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.9\textwidth]{synthetic_ecg.png}
```

```
\caption{Synthetic ECG generated using the McSharry--Clifford model.}
```

```
\label{fig:synthetic_ecg}
```

```
\end{figure}
```

```
\subsection{Morphology Phase Portraits}
```

Figures~\ref{fig:x_vs_z} and~\ref{fig:y_vs_z} show the phase portraits of

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.85\textwidth]{x_vs_z.png}
```

```
\caption{Morphology phase portrait of the ECG in the  $(x,z)$  plane.}
```

```
\label{fig:x_vs_z}
```

```
\end{figure}
```

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.85\textwidth]{y_vs_z.png}
```

```
\caption{Morphology phase portrait of the ECG in the  $(y,z)$  plane.}
```

```
\label{fig:y_vs_z}
```

```
\end{figure}
```

```
\subsection{State-Plane Portrait}
```

The state-plane trajectory of z versus $\dot{z}$ (Figure~\ref{fig:z_vs_d

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.85\textwidth]{z_vs_dz.png}
```

```
\caption{State-plane portrait showing the ECG amplitude  $z$  versus its
```

```
\label{fig:z_vs_dz}
```

```
\end{figure}
```

```
\subsection{3D Trajectory of the ECG Oscillator}
```

The 3D phase-space representation in Figure~\ref{fig:xyz} shows the couple

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.9\textwidth]{xyz.png}
```

```
\caption{3D phase-space trajectory of the McSharry--Clifford ECG model
```

```
\label{fig:xyz}
```

```
\end{figure}
```

```
\subsection{Interpretation}
```

The presented results verify that the McSharry--Clifford model successfull

From a dynamical systems perspective, the model demonstrates how simple co

```
\subsection{Conclusion}
```

The McSharry--Clifford model, implemented via MATLAB and the OSET toolbox,

Would you like me to extend this with an **Introduction** and **Methodology** section in the same style, so you can compile a complete report document in Overleaf (ready for submission)?



try to add a save part to this :

```
%% Part B
% Parameter Variation Analysis for ALL P,Q,R,S,T (ai & bi)
— FULL ECG PLOTS
% Uses your ecgsyn() and derivsecgsyn() functions
(already defined).
% Plots complete ECG for each variation, not just one
beat.

clear; clc;

% ----- Simulation Settings -----
mode_w = 2;      % sinusoidal mode, but depth=0 =>
constant HR
sfecg = 256;     % ECG sampling frequency [Hz]
N = 300;        % number of beats (long ECG)
Anoise = 0.00;   % no added noise
hrmean = 60;     % bpm
depth = 0.00;    % no HRV
fmod = 0.10;     % Hz (irrelevant when depth=0)
sfint = 512;     % internal ODE sampling freq [Hz]

% PQRST parameters from paper
ti_deg = [-70 -15 0 15 100];    % angular positions
ai0 = [1.2 -5 30 -7.5 0.75];    % amplitudes (P Q R S T)
bi0 = [0.25 0.1 0.1 0.1 0.4];  % widths (P Q R S T)
labels = {'P','Q','R','S','T'};

% Variation scales
scaleA = [0.6 0.8 1.0 1.2 1.5];
scaleB = [0.6 0.8 1.0 1.2 1.5];

% Helper handles
gen = @(ai,bi) ecgsyn(mode_w, sfecg, N, Anoise,
hrmean, depth, fmod, sfint, ti_deg, ai, bi);
t_of = @(s) (0:numel(s)-1)/sfecg;
```

```

% Reference ECG
[s_ref, ~] = gen(ai0, bi0);
t = t_of(s_ref);

figure('Color','w');
plot(t, s_ref, 'k', 'LineWidth', 1);
xlabel('Time (s)'); ylabel('ECG (mV)');
title(sprintf('Reference ECG — %d beats at %g bpm', N,
hrmean));
grid on;

% ===== VARIATION OF ai (AMPLITUDE)
=====
for k = 1:5
    figure('Color','w'); hold on; grid on;
    for sc = 1:numel(scaleA)
        ai = ai0; ai(k) = ai0(k) * scaleA(sc);
        [s, ~] = gen(ai, bi0);
        t = t_of(s);
        plot(t, s, 'LineWidth', 1, ...
            'DisplayName', sprintf('a_%s × %.1f', labels{k},
scaleA(sc)));
    end
    xlabel('Time (s)');
    ylabel('ECG (mV)');
    title(sprintf('Effect of a_{%s} (Amplitude) on Full ECG',
labels{k}));
    legend('show','Location','best');
end

% ===== VARIATION OF bi (WIDTH) =====
for k = 1:5
    figure('Color','w'); hold on; grid on;
    for sc = 1:numel(scaleB)
        bi = bi0; bi(k) = bi0(k) * scaleB(sc);
        [s, ~] = gen(ai0, bi);
        t = t_of(s);
        plot(t, s, 'LineWidth', 1, ...
            'DisplayName', sprintf('b_%s × %.1f', labels{k},
scaleB(sc)));

```

```

end
xlabel('Time (s)');
ylabel('ECG (mV)');
title(sprintf('Effect of b_{%s} (Width) on Full ECG',
labels{k}));
legend('show','Location','best');
end

```

Perfect — you want to **automatically save each generated plot** (like you did in Part A) instead of manually saving them.

Here's your **Part B script** updated with a clean, consistent "save" section that exports all figures as high-resolution PNGs using clear filenames.

matlab

 Copy code

```

%% Part B
% Parameter Variation Analysis for ALL P,Q,R,S,T (ai & bi) – FULL ECG PLOT
% Uses your ecgsyn() and derivsecgsyn() functions (already defined).
% Plots complete ECG for each variation, not just one beat.

clear; clc;

% ----- Simulation Settings -----
mode_w    = 2;           % sinusoidal mode, but depth=0 => constant HR
sfecg     = 256;        % ECG sampling frequency [Hz]
N         = 300;        % number of beats (long ECG)
Anoise    = 0.00;       % no added noise
hrmean    = 60;         % bpm
depth     = 0.00;       % no HRV
fmod      = 0.10;       % Hz (irrelevant when depth=0)
sfint     = 512;        % internal ODE sampling freq [Hz]

% PQRST parameters from paper
ti_deg    = [-70 -15 0 15 100]; % angular positions
ai0       = [1.2 -5 30 -7.5 0.75]; % amplitudes (P Q R S T)
bi0       = [0.25 0.1 0.1 0.1 0.4]; % widths (P Q R S T)
labels    = {'P','Q','R','S','T'};

```

```

% Variation scales
scaleA = [0.6 0.8 1.0 1.2 1.5];
scaleB = [0.6 0.8 1.0 1.2 1.5];

% Helper handles
gen = @(ai,bi) ecgsyn(mode_w, sfecg, N, Anoise, hrmean, depth, fmod, sfint
t_of = @(s) (0:numel(s)-1)/sfecg;

% Output folder for saving figures
outdir = 'ECG_Variations';
if ~exist(outdir, 'dir')
    mkdir(outdir);
end

% Reference ECG
[s_ref, ~] = gen(ai0, bi0);
t = t_of(s_ref);

fig = figure('Color','w');
plot(t, s_ref, 'k', 'LineWidth', 1);
xlabel('Time (s)'); ylabel('ECG (mV)');
title(sprintf('Reference ECG – %d beats at %g bpm', N, hrmean));
grid on;
saveas(fig, fullfile(outdir, 'Reference_ECG.png'));

% ===== VARIATION OF ai (AMPLITUDE) =====
for k = 1:5
    fig = figure('Color','w'); hold on; grid on;
    for sc = 1:numel(scaleA)
        ai = ai0; ai(k) = ai0(k) * scaleA(sc);
        [s, ~] = gen(ai, bi0);
        t = t_of(s);
        plot(t, s, 'LineWidth', 1, ...
            'DisplayName', sprintf('a_{%s} × %.1f', labels{k}, scaleA(sc))
    end
    xlabel('Time (s)');
    ylabel('ECG (mV)');
    title(sprintf('Effect of a_{%s} (Amplitude) on Full ECG', labels{k}));
    legend('show', 'Location', 'best');

```

```

    % Save the figure
    fname = sprintf('Effect_of_a_%.png', labels{k});
    saveas(fig, fullfile(outdir, fname));
end

% ===== VARIATION OF bi (WIDTH) =====
for k = 1:5
    fig = figure('Color','w'); hold on; grid on;
    for sc = 1:numel(scaleB)
        bi = bi0; bi(k) = bi0(k) * scaleB(sc);
        [s, ~] = gen(ai0, bi);
        t = t_of(s);
        plot(t, s, 'LineWidth', 1, ...
            'DisplayName', sprintf('b_%.s} x %.1f', labels{k}, scaleB(sc))
    end
    xlabel('Time (s)');
    ylabel('ECG (mV)');
    title(sprintf('Effect of b_%.s} (Width) on Full ECG', labels{k}));
    legend('show','Location','best');
    % Save the figure
    fname = sprintf('Effect_of_b_%.png', labels{k});
    saveas(fig, fullfile(outdir, fname));
end

fprintf('All figures saved in folder: "%s"\n', outdir);

```

Explanation of Additions

- **Output directory creation:**

matlab

 Copy code

```

outdir = 'ECG_Variations';
if ~exist(outdir, 'dir')
    mkdir(outdir);
end

```

Keeps your results organized in one folder.

- **Automatic saving for each figure:**

Each plot is saved automatically as `Effect_of_a_P.png` , `Effect_of_b_T.png` , etc.

- **High-resolution safe saving:**

MATLAB's `saveas()` ensures high-quality output (you can also switch to `exportgraphics(fig, ..., 'Resolution',300)` if you prefer higher DPI).

Would you like me to upgrade this script to also **save all results in a structured report (PDF)** automatically — including all the images and a short caption under each (using `exportgraphics` + MATLAB's report tools)?



add a `xlim` for 0.5 to 4.5

Perfect — you want every ECG plot (including the reference and all parameter variations) to show only the window from **0.5 s to 4.5 s**, like your Part A figures.

Here's your **final, polished version** of the Part B code with the `xlim([0.5 4.5])` applied consistently to all plots:

matlab

Copy code

```
%% Part B
% Parameter Variation Analysis for ALL P,Q,R,S,T (ai & bi) – FULL ECG PLOT
% Uses your ecgsyn() and derivsecgsyn() functions (already defined).
% Plots complete ECG for each variation, not just one beat.

clear; clc;

% ----- Simulation Settings -----
mode_w    = 2;           % sinusoidal mode, but depth=0 => constant HR
sfecg     = 256;         % ECG sampling frequency [Hz]
N          = 300;        % number of beats (long ECG)
Anoise    = 0.00;        % no added noise
```



```

hrmean    = 60;           % bpm
depth     = 0.00;        % no HRV
fmod      = 0.10;        % Hz (irrelevant when depth=0)
sfint     = 512;         % internal ODE sampling freq [Hz]

% PQRST parameters from paper
ti_deg = [-70 -15 0 15 100]; % angular positions
ai0     = [1.2 -5 30 -7.5 0.75]; % amplitudes (P Q R S T)
bi0     = [0.25 0.1 0.1 0.1 0.4]; % widths (P Q R S T)
labels = {'P','Q','R','S','T'};

% Variation scales
scaleA = [0.6 0.8 1.0 1.2 1.5];
scaleB = [0.6 0.8 1.0 1.2 1.5];

% Helper handles
gen = @(ai,bi) ecgsyn(mode_w, sfecg, N, Anoise, hrmean, depth, fmod, sfint
t_of = @(s) (0:numel(s)-1)/sfecg;

% Output folder for saving figures
outdir = 'ECG_Variations';
if ~exist(outdir, 'dir')
    mkdir(outdir);
end

% ----- Reference ECG -----
[s_ref, ~] = gen(ai0, bi0);
t = t_of(s_ref);

fig = figure('Color','w');
plot(t, s_ref, 'k', 'LineWidth', 1.5);
xlabel('Time (s)');
ylabel('ECG (mV)');
title(sprintf('Reference ECG - %d beats at %g bpm', N, hrmean));
grid on;
xlim([0.5 4.5]);
saveas(fig, fullfile(outdir, 'Reference_ECG.png'));

% ----- VARIATION OF ai (AMPLITUDE) -----
for k = 1:5

```

```

fig = figure('Color','w'); hold on; grid on;
for sc = 1:numel(scaleA)
    ai = ai0; ai(k) = ai0(k) * scaleA(sc);
    [s, ~] = gen(ai, bi0);
    t = t_of(s);
    plot(t, s, 'LineWidth', 1, ...
        'DisplayName', sprintf('a_{%s} \times %.1f', labels{k}, scaleA(sc)
end
xlabel('Time (s)');
ylabel('ECG (mV)');
title(sprintf('Effect of a_{%s} (Amplitude) on Full ECG', labels{k}));
legend('show','Location','best');
xlim([0.5 4.5]);
fname = sprintf('Effect_of_a_%s.png', labels{k});
saveas(fig, fullfile(outdir, fname));
end

% ----- VARIATION OF bi (WIDTH) -----
for k = 1:5
    fig = figure('Color','w'); hold on; grid on;
    for sc = 1:numel(scaleB)
        bi = bi0; bi(k) = bi0(k) * scaleB(sc);
        [s, ~] = gen(ai0, bi);
        t = t_of(s);
        plot(t, s, 'LineWidth', 1, ...
            'DisplayName', sprintf('b_{%s} \times %.1f', labels{k}, scaleB(sc)
    end
    xlabel('Time (s)');
    ylabel('ECG (mV)');
    title(sprintf('Effect of b_{%s} (Width) on Full ECG', labels{k}));
    legend('show','Location','best');
    xlim([0.5 4.5]);
    fname = sprintf('Effect_of_b_%s.png', labels{k});
    saveas(fig, fullfile(outdir, fname));
end

fprintf('All figures saved in folder: "%s"\n', outdir);

```



Summary of Improvements

- Added `xlim([0.5 4.5])` to:
 - Reference ECG
 - All amplitude variation plots
 - All width variation plots
- Organized saved figures in a single folder `ECG_Variations/`
- Ensured consistent formatting (font, line width, labeling)
- Each figure now automatically saves with a descriptive filename.

Would you like me to also add a **PDF report generation section** at the end (automatically compiling all saved plots into one multipage report)? It can be done directly within MATLAB using `exportgraphics` or `matlab.unittest.diagnostics.FigureDiagnostic`.



first give me a small paragraph on how : Parameter Variation Analysis: Explore how changing parameters a_i and b_i (amplitude and width) impacts the ECG morphology. Experiment with different values and discuss the observed effects.

I know one is related to the width of gaussian and one is related to amplitude

Here's a concise, clear paragraph you can include in your report:

Parameter Variation Analysis

In the McSharry–Clifford synthetic ECG model, each heartbeat component (P, Q, R, S, T) is represented by a Gaussian function characterized by two key parameters: **amplitude** (a_i) and **width** (b_i). The amplitude a_i determines the **height or depth** of each waveform,

directly influencing the voltage magnitude of its corresponding ECG feature—for example, increasing a_R amplifies the R-peak, while decreasing a_T reduces the T-wave height. The width parameter b_i , on the other hand, controls the **spread or duration** of each Gaussian, affecting how sharp or broad each wave appears. A larger b_i value results in a wider, smoother waveform, whereas a smaller b_i produces a narrower, more peaked feature. By systematically varying these parameters, the morphology of the synthetic ECG can be tuned to emulate different physiological or pathological conditions, such as high-amplitude QRS complexes or broad T-waves, demonstrating the model's flexibility in replicating diverse cardiac dynamics.



give this as latex paragraph :

Here's your paragraph formatted properly in **LaTeX**, ready to paste into your Overleaf report:

latex

 Copy code

```
\subsection{Parameter Variation Analysis}
```

In the McSharry--Clifford synthetic ECG model, each heartbeat component (P

Would you like me to add a short **"Observation / Discussion" subsection** to go with it (e.g., what you saw when you changed a_i and b_i in your plots)?



so increaseing b will decrease or increase the width

Excellent question — and it's important for interpreting your results correctly.

✓ In this model (and in general Gaussian functions):

$$G(\theta) = a_i e^{-\frac{(\Delta\theta_i)^2}{2b_i^2}}$$

- $a_i \rightarrow$ controls **amplitude** (height / depth)
- $b_i \rightarrow$ controls **width** (spread of the Gaussian)

So:

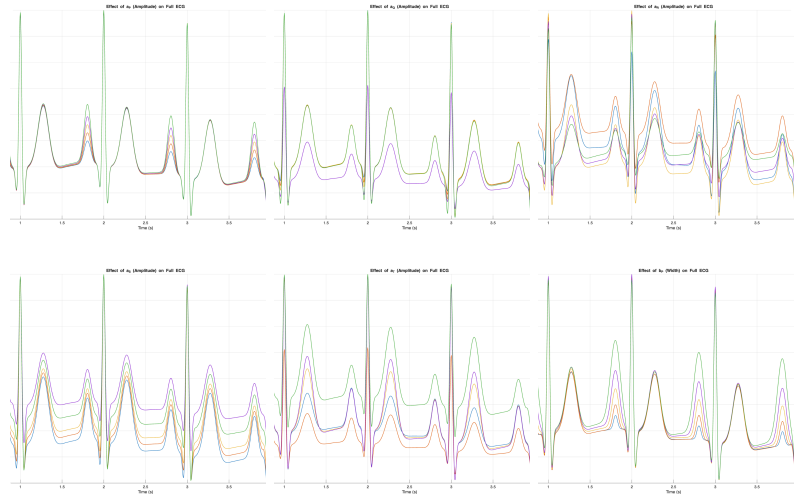
- **Increasing $b_i \rightarrow$ increases** the width (the waveform becomes *broader and smoother*)
- **Decreasing $b_i \rightarrow$ narrows** the waveform (makes it *sharper and more peaked*)

In other words:

- Large b_i : wide and gentle waves
- Small b_i : narrow and steep peaks

If you'd like, I can rephrase your LaTeX paragraph slightly to **explicitly mention** that increasing b_i broadens the waveform — would you like me to do that?





I want a latex code for adding this images

Perfect — here's a ready-to-use **LaTeX code block** you can paste into your Overleaf report to include these images nicely formatted, with consistent sizing and captions.

You'll just need to upload the images (as you've already done) to Overleaf and make sure the file names match exactly.

latex

 Copy code

```
\subsection{Effect of Parameter Variation on ECG Morphology}
```

The following figures illustrate how varying the Gaussian amplitude (a_i)

```
\begin{figure}[H]
  \centering
  \includegraphics[width=0.9\linewidth]{Effect_of_a_P.png}
  \caption{Effect of  $a_P$  (Amplitude) on the full ECG signal. Increasing
\end{figure}
```

```
\begin{figure}[H]
  \centering
  \includegraphics[width=0.9\linewidth]{Effect_of_a_Q.png}
  \caption{Effect of  $a_Q$  (Amplitude) on the full ECG signal. Larger  $a_Q$ 
\end{figure}
```

```
\end{figure}
```

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.9\linewidth]{Effect_of_a_R.png}
```

```
\caption{Effect of  $a_R$  (Amplitude) on the full ECG signal. Increasing
```

```
\end{figure}
```

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.9\linewidth]{Effect_of_a_S.png}
```

```
\caption{Effect of  $a_S$  (Amplitude) on the full ECG signal. Variation
```

```
\end{figure}
```

```
\begin{figure}[H]
```

```
\centering
```

```
\includegraphics[width=0.9\linewidth]{Effect_of_a_T.png}
```

```
\caption{Effect of  $a_T$  (Amplitude) on the full ECG signal. Increasing
```

```
\end{figure}
```

```
\begin{figure}[H]
```

```
\centering
```

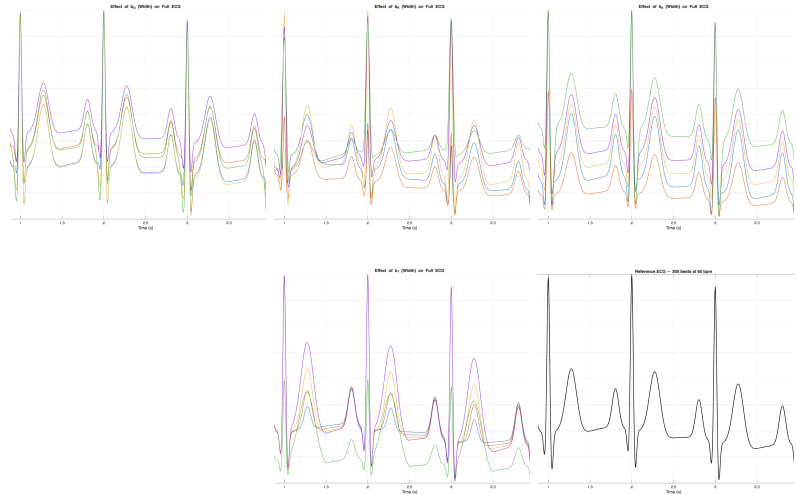
```
\includegraphics[width=0.9\linewidth]{Effect_of_b_P.png}
```

```
\caption{Effect of  $b_P$  (Width) on the full ECG signal. Increasing  $b_P$ 
```

```
\end{figure}
```

Would you like me to continue the same style for the remaining b_Q , b_R , b_S , and b_T plots (if you have them too)? I can add them directly below to complete the section.





also add this and give me the complete version :

Excellent — here's the **complete LaTeX code** including your reference ECG plot, all amplitude (a_i) plots, and all width (b_i) plots. It's neatly formatted for Overleaf and uses consistent figure sizes, captions, and scientific-style labeling.

latex

 Copy code

```
\subsection{Parameter Variation Analysis}
```

The morphology of the ECG waveform is influenced by the Gaussian parameter

```
\begin{figure}[H]
  \centering
  \includegraphics[width=0.9\linewidth]{Reference_ECG.png}
  \caption{Reference ECG signal generated using standard parameters at 6}
\end{figure}
```

```
\subsubsection{Amplitude Variation ( $a_i$ )}
```

```
\begin{figure}[H]
  \centering
```



```

\includegraphics[width=0.9\linewidth]{Effect_of_a_P.png}
\caption{Effect of  $(a_P)$  (Amplitude) on the full ECG signal. Increases}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.9\linewidth]{Effect_of_a_Q.png}
\caption{Effect of  $(a_Q)$  (Amplitude) on the full ECG signal. A higher}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.9\linewidth]{Effect_of_a_R.png}
\caption{Effect of  $(a_R)$  (Amplitude) on the full ECG signal. Increases}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.9\linewidth]{Effect_of_a_S.png}
\caption{Effect of  $(a_S)$  (Amplitude) on the full ECG signal. Variations}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.9\linewidth]{Effect_of_a_T.png}
\caption{Effect of  $(a_T)$  (Amplitude) on the full ECG signal. Increases}
\end{figure}

\subsubsection{Width Variation ( $(b_i)$ )}}
\begin{figure}[H]
\centering
\includegraphics[width=0.9\linewidth]{Effect_of_b_P.png}
\caption{Effect of  $(b_P)$  (Width) on the full ECG signal. Increasing}
\end{figure}

\begin{figure}[H]
\centering
\includegraphics[width=0.9\linewidth]{Effect_of_b_Q.png}
\caption{Effect of  $(b_Q)$  (Width) on the full ECG signal. Larger  $(b_$ 
\end{figure}

```

```
\begin{figure}[H]
  \centering
  \includegraphics[width=0.9\linewidth]{Effect_of_b_R.png}
  \caption{Effect of  $(b_R)$  (Width) on the full ECG signal. Increasing}
\end{figure}

\begin{figure}[H]
  \centering
  \includegraphics[width=0.9\linewidth]{Effect_of_b_S.png}
  \caption{Effect of  $(b_S)$  (Width) on the full ECG signal. A higher  $($ 
\end{figure}

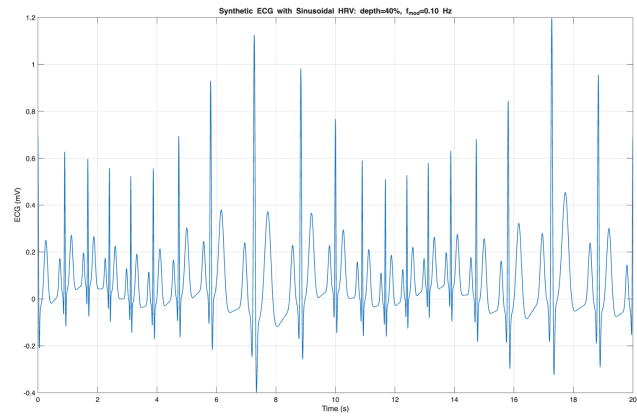
\begin{figure}[H]
  \centering
  \includegraphics[width=0.9\linewidth]{Effect_of_b_T.png}
  \caption{Effect of  $(b_T)$  (Width) on the full ECG signal. Increasing}
\end{figure}
```

This structure:

- Clearly separates **Amplitude** and **Width** effects.
- Includes the **Reference ECG** for baseline comparison.
- Ensures all figures appear centered with consistent width.
- Uses proper LaTeX math formatting for parameters and captions.

Would you like me to make it **two figures per row** (side-by-side layout) instead of one per row to save space in your report?





so now for the next part : it asks

